

R Lab7. Análisis de datos de microarrays con R

Bioinformática. Ingeniería Biomédica

Los microarrays son una de las herramientas más populares para comprender los fenómenos biológicos mediante mediciones a gran escala de muestras biológicas, normalmente DNA, RNA o proteínas. Esta técnica se ha utilizado para una gran variedad de objetivos, que van desde obtener el perfil de expresión génica hasta la identificación de biomarcadores, o para comprender las relaciones entre los genes y sus funciones biológicas.

Existen diferentes tecnologías de microarray: Affymetrix, Illumina, etc. En este laboratorio nos centramos en los microarrays de expresión de Affymetrix. La mayoría de los métodos utilizados se pueden utilizar en otras plataformas con pequeñas modificaciones.

Obtener datos de repositorios públicos

Fundamentalmente existen dos repositorios para obtener datos de experimentos llevados a cabo con microarrays: GEO y ArrayExpress.

La plataforma más utilizada es GEO. Los distintos conjuntos de datos disponibles tienen un identificador, por ejemplo ‘GSE24460’ (<https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE24460>), cuyo objetivo fue medir la resistencia a un fármaco quimioterápico y que contiene la expresión de 22277 genes en cuatro muestras (arrays), dos de cada tipo: casos y controles. La plataforma utilizada fue Affymetrix Human Genome U133A 2.0 Array.

Podemos utilizar el paquete `GEOquery`, disponible en Bioconductor, para obtener un *dataset* directamente del repositorio GEO. Instalamos y cargamos este paquete.

```
if(!("GEOquery" %in% installed.packages()))  
  BiocManager::install("GEOquery")  
library("GEOquery")
```

Leemos un *dataset* concreto con la función `GEOquery::getGEO()`, pasándole como argumento el identificador del dataset como *string*. Debemos utilizar el argumento `AnnotGPL = TRUE` para flexibilizar la información sobre las anotaciones de los genes. Por ejemplo, para obtener los datos con el identificador “GSE24460” y almacenarlos en el objeto `geo`.

```
geo <- getGEO("GSE24460", AnnotGPL = TRUE)  
class(geo)
```

También es posible obtener datos de *ArrayExpress* a partir de la función `ArrayExpress()` del paquete `ArrayExpress` disponible en Bioconductor.

En el material complementario de este laboratorio encontraréis tanto la información completa como los pasos para realizar el análisis de bajo nivel o procesado.

Análisis de expresión diferencial

Buscamos genes que se expresen de manera diferencial entre tipos de muestras. El punto de partida será la matriz de expresión. Vamos a utilizar el paquete `limma` disponible en Bioconductor, uno de los más populares para llevar a cabo este análisis basado en modelos lineales. Utilizamos unos datos sobre cáncer de colon disponibles en el paquete `antiProfilesData`. El *dataset* se llama `apColonData` y se trata de un objeto de la clase `ExpressionSet` con información sobre 68 muestras y 5339 genes cuyos niveles de expresión ya están pre-procesados. La información sobre si la muestra es cancerosa o no se recoge en el campo `Status` y se codifica como 1 o 0 respectivamente. Podemos acceder a estas variables fenotípicas utilizando la función `pData()`.

1. Instala y carga los paquetes necesarios.

1.1 Carga el objeto `apColonData` y construye una tabla de frecuencias para la variable fenotípica `Status` que contiene la información de si la muestra es o no cancerosa.

- Para cargar los datos utilizamos la función `data()`
- Con el método `pData()`, se obtienen las variables fenotípicas

2. El método *limma* que se utiliza para identificar los genes diferencialmente expresados consiste en ajustar un modelo lineal para cada gen utilizando la información de todos los arrays, y posteriormente utilizar el estadístico *empirical Bayes* que es una modificación del test t-Student. Calculamos los estadísticos junto con los correspondientes p-valores asociados al contraste de igualdad de expresión media y los log *fold-changes* para ayudarnos en la interpretación de los resultados. Seguimos los siguientes pasos:

2.1. El primer paso para llevar a cabo un análisis de expresión diferencial con `limma` es definir una matriz de diseño que recoja los detalles del diseño experimental. Para ello utilizamos la función `model.matrix()`. Debemos definir como factor la variable fenotípica que identifica el tipo de muestra, en este caso `Status`.

```
design <- model.matrix(~0 + factor(pData(apColonData)$Status))
```

¿De qué clase es el objeto `design`? Interpreta la información que contiene.

2.2. Una vez definida la matriz de diseño, ajustamos el modelo lineal en cada gen con la función `lmFit()`. Esta función necesita de dos argumentos: el objeto `ExpressionSet` que contiene la matriz de expresión y la matriz de diseño. ¿Qué tipo de objeto obtienes con `lmFit()`?

2.3. El estadístico *empirical Bayes moderated t-test* lo aplicamos con la función `eBayes()`. El argumento de esta función es el objeto `MArrayLM` creado en el paso anterior. ¿Qué tipo de objeto obtienes ahora?

2.4. Podemos utilizar la función `topTable()` para ordenar los genes diferencialmente expresados de mayor a menor significación. Esta función requiere los siguientes argumentos:

- el objeto `MArrayLM` creado en el paso anterior.
- `adjust`, en el que se indica el método a utilizar para resolver el problema de las comparaciones múltiples. Utiliza el método “fdr”.
- `sort.by` en el que se indica un campo para ordenar los genes según la cantidad de expresión diferencial. Un valor común es “B”, que indica el valor absoluto de la diferencia de expresión.
- `number` que indica el número de genes a incluir en la tabla. Con `number=Inf` se obtendrá la tabla para todos los genes analizados.

Obtén la tabla para estos datos.

2.5. A partir de la tabla creada en el punto anterior, podemos seleccionar los genes diferencialmente expresados. El criterio para hacer esta selección puede variar según los objetivos concretos que se tengan en cada estudio. En este caso, selecciona aquellos que tienen un p-valor ajustado menor que 0.0001 y un log *fold change* mayor que 2. ¿Cuántos genes están diferencialmente expresados según este criterio? Obtén la lista de los 10 genes más diferencialmente expresados.

El problema cuando tenemos múltiples estados

En la sección anterior hemos visto como obtener los genes diferencialmente expresados entre dos condiciones experimentales. Sin embargo, es posible que diseñemos nuestros experimentos con más de dos condiciones y necesitemos comparar más de dos grupos. Para verlo con un ejemplo, vamos a considerar la variable fenotípica `SubType` que tiene cuatro niveles: adenoma, cancer colorectal, tumor y normal (sin cancer).

1. Define la matriz de diseño a partir de esa variable, cuyos niveles son:

```
table(pData(apColonData)$SubType)
```

```
##
##          adenoma colorectal_cancer          normal          tumor
##              15              15              15              23
```

2. Ajusta los modelos lineales a cada gen con la función `lmFit()`.

2.1. Para definir los pares de niveles a comparar se utiliza la función `makeContrasts()`. Esta función construye una matriz de contraste. Para comparar los tres tipos de cáncer con los individuos sanos escribimos,

```
contrast.matrix <- makeContrasts(adenoma-normal,colorectal_cancer-normal,tumor-normal,
                                levels=c(colnames(design2)))
contrast.matrix
```

```
##          Contrasts
## Levels          adenoma - normal colorectal_cancer - normal tumor - normal
## adenoma              1              0              0
## colorectal_cancer      0              1              0
## normal               -1             -1             -1
## tumor                 0              0              1
```

Utiliza la función `contrasts.fit()` para ajustar los modelos definitivos. Esta función tiene como primer argumento un objeto `MArrayLM` con los modelos lineales ajustados a cada gen y como segundo argumento la matriz de contraste.

2.2. El estadístico empírico Bayes lo aplicamos con la función `eBayes()` sobre los modelos definitivos.

Para obtener los genes diferencialmente expresados debemos elegir el contraste de interés. Utilizaremos el argumento `coef` de la función `topTable()` para ordenar los resultados según ese contraste. Ordenar los genes según la comparación de los adenomas con los normales (el primer contraste).

2.3. Selecciona los genes diferencialmente expresados utilizando el criterio: significativos a nivel 0.0001 y con logFC mayor que 2. ¿Cuántos genes tienen una expresión diferencial en los adenomas respecto de los individuos sanos?

Volcano plot

Una forma muy habitual de presentar los resultados de un análisis de expresión diferencial es el *Volcano Plot*. Es un gráfico de dispersión en el que se representa la significación de cada gen frente al *fold change*. Utilizamos el objeto `tested2` que contiene la información de los genes diferencialmente expresados en el adenoma para hacer esta representación.

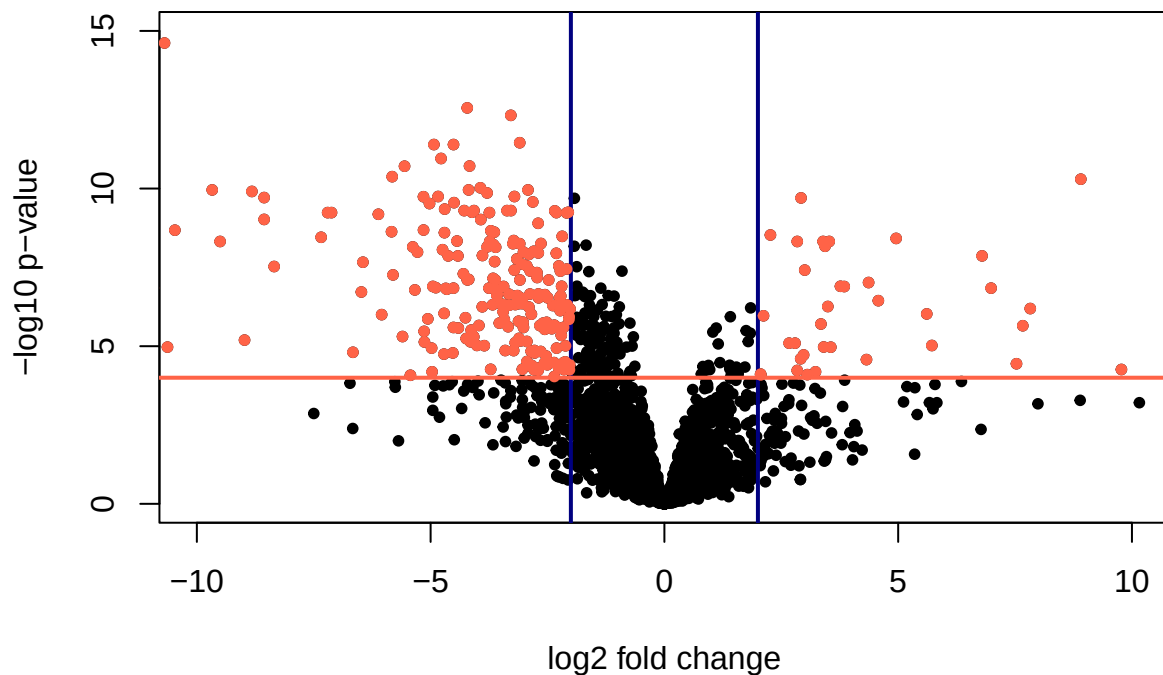
```

plot(tested2$logFC, -log10(tested2$adj.P.Val),pch=20,
     xlim=c(-10, 10), ylim=c(0, 15),
     xlab="log2 fold change", ylab="-log10 p-value")

abline(v=c(-2,2),lwd=2,col="navy")
abline(h=-log10(0.0001),lwd=2, col="tomato")

# en rojo los genes DE
points(tested2$logFC[is.element(rownames(tested2),rownames(DE2))],
       -log10(tested2$adj.P.Val[is.element(rownames(tested2),rownames(DE2))]),
       pch=20,col="tomato")

```



La función `volcanoplot()` disponible en el paquete `limma` permite hacer este gráfico, pasándole como argumento un objeto de tipo `MArrayLM`. Utiliza esta función para hacer el gráfico a partir del objeto `fitE2` creado en el punto 2.2.

Enriquecimiento funcional

Como resultado de un análisis de expresión diferencial obtenemos una lista de genes que podemos enriquecer con los métodos que vimos en el análisis de grupos de genes.

Análisis exploratorio utilizando análisis de componentes principales (PCA)

Estamos midiendo la expresión de 5339 genes en 68 muestras. Cada uno de esos genes es una variable en un espacio multidimensional. Para tratar de reducir las dimensiones del problema y explicar posibles patrones en los datos, podemos tratar de construir combinaciones de las variables que expliquen la máxima variabilidad posible. El análisis de componentes principales (PCA) permite llevar a cabo esto, tratando de encontrar componentes ortogonales que representen los datos lo mejor posible. Cada componente es una combinación lineal de las variables originales donde se proyectan los datos en un espacio de menor dimensión.

El punto de partida es la matriz de expresión ya pre-procesada.

1. Crea un objeto con la matriz de expresión del a partir del objeto `ExpressionSet` de la sección anterior.
2. Podemos utilizar la función genérica `prcomp()` para obtener las componentes principales. Utiliza el método `summary()` con el objeto creado con `prcomp()` para obtener un resumen del PCA. ¿Cuántas componentes se obtienen? ¿con cuántas te quedarías?

Queremos que cada componente principal sea una combinación lineal de genes, dado que la matriz de expresión tiene los genes por filas y las muestras por columnas, debemos transponerla con la función `t()`.

3. Vamos a representar las dos primeras componentes para visualizar algún patrón en los datos. Las coordenadas de cada gen en las componentes que se han construido se almacenan en `myPca$x`. Construye un diagrama de dispersión para representar las dos primeras componentes, utilizando distintos colores para cada tipo de muestra. Interpreta el resultado.

Análisis cluster con datos de microarrays

Tratamos de agrupar juntos genes similares en cuanto a sus niveles de expresión y diferentes a los genes de otros grupos. Así, cuando varios genes se agrupan juntos significa que tienen un patrón de expresión similar en las condiciones analizadas. Vamos a hacer clustering jerárquico con los datos de expresión de los genes diferencialmente expresados en los datos sobre cáncer de colon utilizados anteriormente y almacenados en el objeto `DE`. Utilizamos la función `exprs()` para obtener la matriz de expresión.

```
apColon.DE<-exprs(apColonData)[rownames(DE),]
```

1. Para hacer un análisis cluster debemos definir una medida de la distancia o disimilaridad. Vamos a basar nuestro agrupamiento en la correlación, de forma que serán similares aquellos genes cuyos niveles de expresión estén más relacionados entre sí. Definimos la distancia como la inversa de esta correlación.

Crea una función que reciba una matriz de expresión (denominada `x`), calcule la matriz de correlaciones usando `cor()`, transforme los valores en distancias ($1 - \frac{\rho}{2}$) y devuelva un objeto de clase `dist` utilizando `as.dist()`.

Importante: La matriz debe transponerse con `t()` porque `cor()` calcula correlaciones entre columnas, y en una matriz de expresión, los genes suelen estar en filas.

2. Una vez calculada la matriz de distancias, el siguiente paso es aplicar un método de agrupamiento. En este caso, utilizaremos el *clustering* jerárquico aglomerativo, una técnica ampliamente usada para agrupar elementos según su similitud de forma progresiva.

Vamos a definir una función que:

- Recibe como argumento una matriz de distancias (de clase `dist`).
- Aplica la función `hclust()`, que construye un dendrograma jerárquico.

- Usa como método de enlace (*linkage*) el completo ("complete"), que mide la distancia entre dos clústeres como la distancia máxima entre sus elementos.

3. Para representar gráficamente los resultados, utilizaremos la función `heatmap.2()` del paquete `gplots`. Esta función permite construir mapas de calor con dendrogramas, lo que resulta especialmente útil para explorar patrones de expresión y agrupamientos simultáneamente.

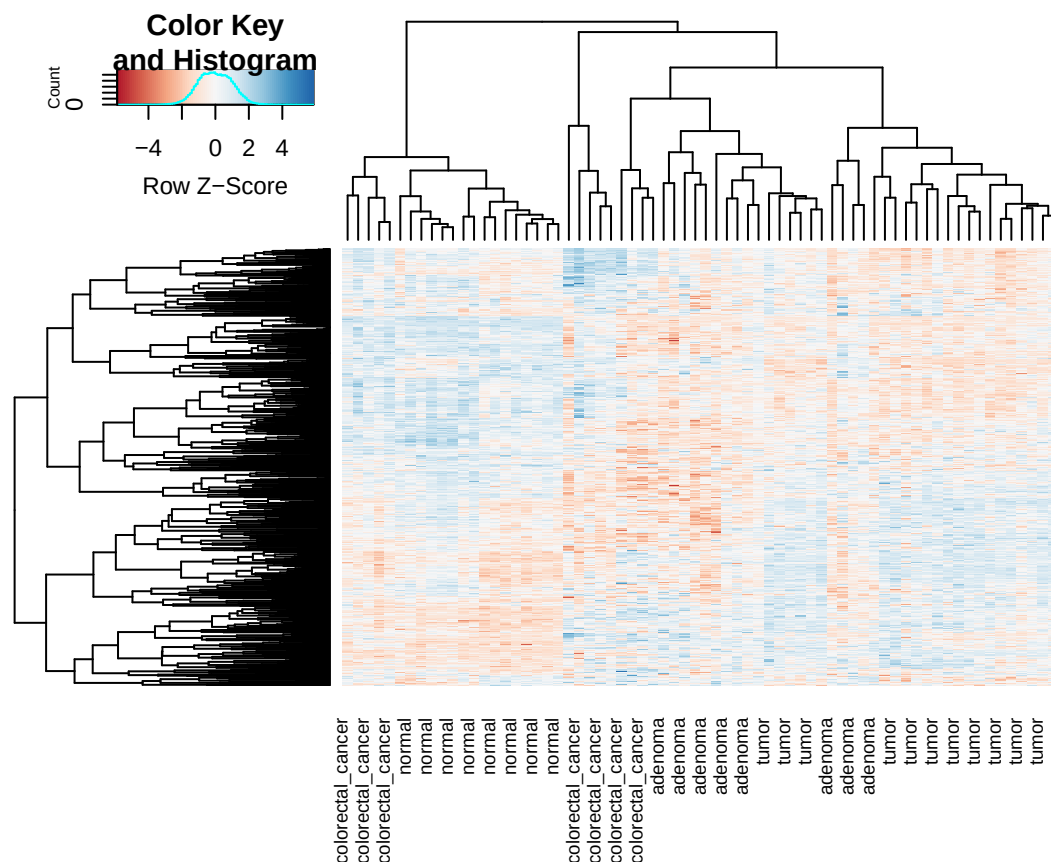
Además, utilizaremos el paquete `RColorBrewer` para seleccionar paletas de colores que mejoren la visualización.

Ambos paquetes están disponibles en CRAN, por lo que puedes instalarlos con:

```
if(!("gplots" %in% installed.packages()))
  install.packages("gplots")
if(!("RColorBrewer" %in% installed.packages()))
  install.packages("RColorBrewer")
library("gplots")
library("RColorBrewer")
```

Construimos el mapa de calor:

```
hp<-heatmap.2(apColon.DE,
  trace = "none",           # sin líneas entre celdas
  scale = "row",           # centramos cada gen (fila)
  col = colorRampPalette(brewer.pal(9, "RdBu"))(100), # paleta de colores
  dendrogram = "both",     # dendrogramas en filas y columnas
  labCol=pData(apColonData)$SubType, # etiquetas para las muestras
  labRow=FALSE,            # sin etiquetas en filas
  Colv = TRUE, Rowv = TRUE, # aplicar clustering a filas y columnas
  margins = c(6, 6),      # márgenes para etiquetas
  cexRow = 0.7, cexCol = 0.7, # tamaño de texto
  hclust=clust.fun,       # definición de método
  distfun=dist.fun)      # definición de distancia
```



Si queremos obtener el listado de genes que pertenecen a cada cluster podemos utilizar la función genérica `cutree()`.

```
clusters <- cutree(as.hclust(hp$rowDendrogram),k=10) # 10 grupos
table(clusters)
```

```
## clusters
## 1 2 3 4 5 6 7 8 9 10
## 180 16 40 130 56 128 47 194 101 20
```

```
# genes del cluster 2
names(clusters)[clusters==2]
```

```
## [1] "208113_x_at" "212198_s_at" "224926_at" "235433_at" "212194_s_at"
## [6] "231876_at" "209071_s_at" "218353_at" "209070_s_at" "234351_x_at"
## [11] "201522_x_at" "231606_at" "212642_s_at" "206042_x_at" "1555725_a_at"
## [16] "203881_s_at"
```